

A Project Report
on
Voice-Driven Desktop Automation & Web Assistant Platform

Submitted by
SIDDHI PANCHAL (2303031080215)
BHAVYA PAREKH (2303031080216)
ARMY CHOTHANI (2403031087135)
KRUPALI THAKOR (2303031080250)

Under the Guidance of
Mr. Raunak Raj
Assistant Professor

In partial fulfilment for the award of the degree of
BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY



Department of Information Technology
Parul Institute of Engineering and Technology, Parul University
Vadodara, Gujarat

Academic Year 2026–2027
Group No.: 86

CERTIFICATE

This is to certify that the project report titled “Voice-Driven Desktop Automation & Web Assistant Platform”, submitted by **Siddhi Panchal (2303031080215)**, **Bhavya Parekh (2303031080216)**, **Armi Chothani (2403031087135)** and **Krupali Thakor (2303031080250)** to Parul University, Vadodara, Gujarat, is a genuine record of the project work carried out by them under my supervision and guidance. The work is found to be satisfactory and is worthy of consideration for the award of the degree of Bachelor of Technology in Information Technology of the University. Place: Vadodara

Date:

Supervisor
Mr. Raunak Raj
Assistant Professor

Project Coordinator
Ms. Dhenuka Patel
Assistant Professor

Head of Department
Dr. Pooja Sapra
Professor and Head,
Department of Information
Technology

DECLARATION

We hereby declare that the project report entitled “Voice-Driven Desktop Automation & Web Assistant Platform”, submitted to the Department of Information Technology, Parul Institute of Engineering and Technology, Parul University, Vadodara, in partial fulfilment of the requirements for the degree of Bachelor of Technology in Information Technology, is a record of our own work carried out under the guidance of Mr. Raunak Raj, Assistant Professor.

The matter embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma. Wherever the work of others has been used, it has been acknowledged through citations and the list of references.

Place: Vadodara

Date:

Siddhi Panchal (2303031080215)

Signature: _____

Bhavya Parekh (2303031080216)

Signature: _____

Armi Chothani (2403031087135)

Signature: _____

Krupali Thakor (2303031080250)

Signature: _____

ACKNOWLEDGEMENT

Behind every successful project lies the contribution, guidance and encouragement of many individuals. We take this opportunity to express our sincere gratitude to all those who supported and guided us during the completion of this project.

First and foremost, we would like to express our heartfelt gratitude to our respected guide, Mr. Raunak Raj (Assistant Professor), for his continuous support, valuable guidance and encouragement throughout the development of this project. His insightful suggestions and constructive feedback helped us overcome many challenges, and working under his supervision has been a great learning experience.

We also express our sincere thanks to Dr. Pooja Sapra (Professor and Head, Department of Information Technology) for her guidance, encouragement and support. Her leadership played an important role in the successful completion of this project.

We would like to thank all the faculty members of the Department of Information Technology for their support throughout our academic journey. Finally, we express our gratitude to our parents, friends and well-wishers for their continuous encouragement, and to the Almighty for the strength and patience to complete this work.

ABSTRACT

VOICE is a browser-based prototype that lets a user control a desktop computer by speech. Keyboard and mouse remain the main way of operating a desktop, and existing voice assistants are mostly built for phones and smart speakers or are tied to one operating system. VOICE addresses this gap for a small, well-defined set of tasks. The frontend is built with React and captures speech through the browser's Web Speech API, which converts it to text. The text is sent to a Python Flask backend, which identifies the user's intent using rule-based keyword and pattern matching (no machine-learning model is used) and executes a predefined action such as opening an application, launching a system utility or running a web search. A text-to-speech engine and an on-screen panel report the outcome to the user. This report describes the requirements, architecture, UML and data-flow diagrams, and a test plan of twenty test cases. The prototype demonstrates the complete path from spoken command to executed action and spoken reply for the supported command set. Quantitative measures such as recognition accuracy and response time were not benchmarked and are left as future work, together with offline recognition, a learned language-understanding model and multi-language support.

Keywords: voice assistant, speech recognition, desktop automation, React, Flask, text-to-speech, rule-based intent recognition.

TABLE OF CONTENTS

CERTIFICATE	ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF TABLES.....	viii
LIST OF FIGURES.....	ix
LIST OF ABBREVIATIONS	x
CHAPTER 1 – INTRODUCTION.....	1
1.1 Problem Statement	1
1.2 Motivation.....	2
1.3 Aim and Objectives.....	2
1.4 Scope and Out-of-Scope	3
1.5 Organisation of the Report.....	3
CHAPTER 2 – LITERATURE REVIEW	4
2.1 Introduction.....	4
2.2 Summary of Research Papers	4
2.3 Critical Evaluation	5
2.4 Summary of the Literature	5
2.5 Limitations of Existing Systems	6
CHAPTER 3 – PROBLEM DEFINITION AND REQUIREMENT ANALYSIS.....	7
3.1 Research Methodology	7
3.2 Proposed Methodology	8
3.3 System Analysis.....	8
3.4 System Requirement Study.....	10
CHAPTER 4 – DESIGN AND IMPLEMENTATION	12
4.1 System Architecture.....	12
4.2 Use-Case Diagram	13
4.3 Activity Diagram	14

4.4 Sequence Diagram	15
4.5 Data Flow Diagrams	16
4.6 Command Interpretation	18
4.7 Implementation Environment	18
4.8 Front-End Implementation	18
4.9 Back-End Implementation	19
4.10 User Interface	19
4.11 Safety Measures in the Implementation	20
CHAPTER 5 – TESTING AND DEPLOYMENT	21
5.1 Testing Approach	21
5.2 Test Environment	21
5.3 Test Cases	21
5.4 Test Result Summary	22
5.5 Deployment	22
5.6 Maintenance	23
CHAPTER 6 – ANALYSIS AND RESULTS	24
6.1 Introduction	24
6.2 Functional Analysis	24
6.3 Comparison with an Existing Tool	24
6.4 Limitations	24
6.5 Accessibility	25
6.6 Done and Planned	25
CHAPTER 7 – CONCLUSION AND FUTURE ENHANCEMENTS	26
7.1 Conclusion	26
7.2 Future Enhancements	26
CHAPTER 8 – REFERENCES	27

LIST OF TABLES

Table 2.1: Summary of reviewed research papers.....	4
Table 2.2: Limitations of existing tools and the approach of VOICE.....	6
Table 3.1: Functional requirements and their use-case and test-case mapping.....	8
Table 3.2: Non-functional requirements.....	9
Table 3.3: Supported voice commands of the prototype	9
Table 3.4: Hardware requirements (single computer running both frontend and backend)	10
Table 3.5: Technology stack and versions	10
Table 3.6: Operating system support status.....	11
Table 4.1: Worked examples from utterance to action.....	18
Table 4.2: Backend components and responsibilities.....	19
Table 5.1: Test environment.....	21
Table 5.2: Test cases for the VOICE prototype.....	21
Table 6.1: Comparison of Windows Voice Access and VOICE.....	24
Table 6.2: Status of features	25

LIST OF FIGURES

Figure 4.1: System architecture of VOICE	12
Figure 4.2: Use-case diagram of VOICE.....	13
Figure 4.3: Activity diagram of a voice command.....	14
Figure 4.4: Sequence diagram of the command flow	15
Figure 4.5: Data flow diagram, level 0	16
Figure 4.6: Data flow diagram, level 1	17
Figure 4.7: Main screen of the VOICE interface.....	19
Figure 4.8: Recognised command and system response	19

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
FR	Functional Requirement
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
NER	Named Entity Recognition
NFR	Non-Functional Requirement
NLP	Natural Language Processing
NLU	Natural Language Understanding
OS	Operating System
REST	Representational State Transfer
SLU	Spoken Language Understanding
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
VA	Virtual Assistant
VS Code	Visual Studio Code
WER	Word Error Rate

CHAPTER 1 – INTRODUCTION

Speaking to a computer is now an everyday habit on phones and smart speakers, yet most desktop work is still done with a keyboard and a mouse. This report presents VOICE, a browser-based prototype that lets a user speak a short command, such as “open notepad” or “search python tutorials on google”, and have the computer carry it out and reply aloud. The system combines a React user interface, the browser’s Web Speech API for speech-to-text, a Python Flask backend that interprets the command, and a text-to-speech engine for the reply. This chapter states the problem, the objectives and the boundaries of the work.

1.1 Problem Statement

Voice assistants such as Siri, Alexa and Google Assistant have made spoken interaction familiar, and research on the underlying speech and language technology is very active [1], [2], [3]. These assistants are, however, designed mainly around smartphones and smart speakers. On a desktop computer, routine actions such as opening an application, starting a system utility or searching the web still take several clicks or key presses and interrupt what the user is doing.

Voice control does exist on the desktop, but the options are limited. Windows Voice Access can operate the Windows 11 interface by voice, but it is a Windows-only feature available from Windows 11 version 22H2 [4], and Microsoft’s Cortana app, which earlier offered voice commands, is no longer supported in Windows [5]. Such closed products cannot be inspected to see how a spoken sentence becomes an action, nor easily extended by a student or small team with a new command.

Problem definition. There is no simple, open and easily extensible tool that lets a desktop user launch applications, open system utilities and run web searches by speaking, and that gives spoken feedback, without depending on a particular vendor’s assistant. VOICE addresses this by building a small browser-based assistant on widely available web technologies. It recognises speech in the browser, identifies the intent with transparent rule-based matching, executes a predefined action on the host computer, and reports the outcome by voice and on screen.

1.2 Motivation

The motivation for VOICE is practical and educational. First, the team wanted to see how far ordinary web technologies (React, Flask and the browser's built-in speech features) can be combined to control a desktop without special hardware or paid speech services. Second, the project brings several topics of the B.Tech curriculum together in one working system: speech-to-text, basic natural language processing [6], client-server communication through a REST interface, and systematic software testing [7]. Third, a rule-based design keeps every step visible, which makes it a suitable base for later replacing the intent matcher with a learned model of the kind reviewed in Chapter 2. Finally, hands-free control may help people who find keyboards and mice difficult to use; this is treated in this report only as a potential benefit, because no user study has been carried out.

1.3 Aim and Objectives

Aim: to design and build a prototype browser-based voice assistant that executes a predefined set of desktop and web commands from spoken input and reports the result by voice and on screen.

The objectives are:

- 1. Capture spoken commands through the browser microphone and convert them to text using the Web Speech API in Google Chrome or Microsoft Edge.
- 2. Support the nine command patterns listed in Table 3.3: opening four applications, launching two system utilities and three web actions.
- 3. Identify the intent of the recognised text in a Flask backend using rule-based keyword and pattern matching, and return the intent, parameters and a response message as JSON.
- 4. Execute the identified action on the host computer through Python, with Windows as the primary tested operating system.
- 5. Give feedback both on screen and by speech using a text-to-speech engine.
- 6. Handle unsupported or unclear input safely: report “command not understood” and perform no action.
- 7. Aim to show the response within 2 seconds of the end of the utterance (a design target; measuring it is future work).

- **8.** Verify the prototype with a documented set of 20 test cases (Table 5.2) and report the outcome honestly, including limitations.

1.4 Scope and Out-of-Scope

In scope:

- A browser-based interface for one user on one computer (localhost).
- English voice commands, limited to the nine patterns in Table 3.3.
- Rule-based intent recognition, execution of predefined actions, and spoken plus on-screen feedback.
- Windows as the primary tested operating system.

Out of scope:

- Wake-word detection and always-listening mode; multi-turn conversation and context.
- Machine-learning or deep-learning intent models.
- Languages other than English; offline speech recognition.
- Operating systems other than Windows (macOS and Linux are untested).
- User accounts, login, an administrator module and command history storage.
- Hosting on a public server; performance benchmarking; evaluation with noisy speech, accents or users with disabilities.

Security limits. VOICE is a prototype for a trusted, single-user machine. It executes only the predefined actions of Table 3.3 and does not pass the user's text to a system shell, but it has no authentication. In Chrome the speech-recognition service sends the audio to an external server [8], so microphone input is not private. The backend should not be exposed to a network.

1.5 Organisation of the Report

Chapter 2 reviews related research. Chapter 3 defines the problem and the requirements. Chapter 4 presents the design and implementation. Chapter 5 describes testing and deployment. Chapter 6 discusses the results and limitations. Chapter 7 concludes and lists future enhancements, and Chapter 8 lists the references.

CHAPTER 2 – LITERATURE REVIEW

2.1 Introduction

A voice assistant combines automatic speech recognition (ASR), natural language understanding (NLU), action execution and speech synthesis [6]. This chapter reviews six peer-reviewed papers on recognition and understanding in voice assistants. All six come from industrial research groups and describe large-scale systems; VOICE is a small prototype that reuses an existing recogniser, so the papers are used to understand the state of the art and to place the limits of VOICE, not as designs to copy.

2.2 Summary of Research Papers

Table 2.1 summarises the reviewed papers. Each was checked against the publisher's or the authors' listing.

Table 2.1: Summary of reviewed research papers

Ref.	Authors, year	Title	Key finding
[1]	Saebi et al., 2021	A discriminative entity-aware language model for virtual assistants	Knowledge-graph features added to a discriminative n-gram language model; over 25% relative reduction in sentence error rate on some synthesised test sets with less popular entities.
[2]	Mitra et al., 2021	Analysis and tuning of a voice assistant system for dysfluent speech	Tuning decoding parameters of an existing recogniser improved intended-speech word error rate by 24% (relative) for people who stutter.
[9]	Muralidharan et al., 2021	DEXTER: Deep encoding of external knowledge for named entity recognition in virtual assistants	External knowledge for noisy voice-assistant input gave about a 6% reduction in NER error rate and up to 5% on semantic parsing.
[3]	Radfar et al., 2021	FANS: Fusing ASR and NLU for on-device SLU	End-to-end model predicting intent and slots directly from audio, without an intermediate transcript.
[10]	Van Gysel et al., 2022	Space-efficient representation of entity-centric query language models	Compact language models for template-plus-entity queries (e.g. media requests); studies size versus perplexity and word error rate.

2.3 Critical Evaluation

[1] Recognition errors on names are a major weakness and can be reduced with outside knowledge. The gap is that the method needs a knowledge graph and lattice rescoring inside the ASR pipeline. VOICE avoids the problem by design because it uses a fixed list of applications and websites, at the cost of flexibility.

[2] Recognisers trained mainly on fluent speech perform worse on repetitions and prolongations. The results are for one consumer system and 18 participants. VOICE relies on the browser's recogniser and has not been tested with atypical speech, so any accessibility benefit remains potential.

[9] Input to language understanding in an assistant is noisy because of recognition errors. The approach uses proprietary data. VOICE's keyword rules are tolerant of filler words but cannot repair a misrecognised word.

[3] Intent and slots can be predicted directly from audio, which suits on-device use. Such models need large training data. VOICE uses a simpler two-step pipeline (browser STT, then rules), which is transparent but cannot recover from recognition errors.

[10] Real assistants treat many requests as templates with an entity slot, which is essentially how VOICE's "search ... on google" pattern works. The paper targets very large entity lists, whereas VOICE has nine patterns.

[11] Labelled multilingual utterances exist and could train a learned NLU. The dataset covers general assistant domains, not desktop automation. VOICE is English-only with hand-written rules; a future version could fine-tune a model on such data plus desktop-command examples.

2.4 Summary of the Literature

- Recognition accuracy for names, atypical speech and noisy input is an open concern [1], [2], [9].
- Intent and slot prediction is moving toward end-to-end and on-device models [3], [10].
- Multilingual coverage and labelled data are growing [11].

- None of the six papers addresses browser-based desktop automation with local execution of commands; they study components of large pipelines. This is the gap VOICE targets at prototype scale.

2.5 Limitations of Existing Systems

Table 2.2: Limitations of existing tools and the approach of VOICE

Existing tool	Limitation	Our approach
Smartphone and smart-speaker assistants (Google Assistant, Siri, Alexa)	Designed mainly around phones and speakers; desktop support, where it exists, is tied to one vendor's platform and fixed by the vendor.	Runs in a desktop browser and launches local applications through its own backend.
Cortana (Windows app)	Standalone Windows support ended in 2023 [5].	Does not depend on a discontinued product.
Windows Voice Access	Windows 11 (22H2 or later) only; broad, closed interface control rather than a small customisable command set [4].	Small, transparent rule set that can be edited and extended.
Research systems reviewed in Section 2.2	Study components (recognition, understanding) inside large industrial pipelines.	Small but complete voice-to-action pipeline for the desktop.

CHAPTER 3 – PROBLEM DEFINITION AND REQUIREMENT ANALYSIS

3.1 Research Methodology

VOICE was developed with an incremental prototype approach [7]: requirements were analysed first, then the design was prepared, the frontend and backend were implemented step by step, and each increment was tested before the next was added.

3.1.1 Working

- The user opens the VOICE page in Chrome or Edge, allows microphone access and clicks the microphone button.
- The browser's Web Speech API converts the speech to text [8]. In Chrome this uses a server-based service, so an internet connection is required.
- The React frontend sends the recognised text to the Flask backend as a JSON request.
- The backend normalises the text (lower case, removal of punctuation and filler words) and matches it against keyword rules to find the intent and its parameters.
- If the intent is recognised, the backend executes the predefined action with Python (subprocess for applications and utilities, webbrowser for web pages) [12].
- The backend returns a response message, which the UI displays and the text-to-speech engine pyttsx3 speaks [13]. If the intent is not recognised, the reply is "command not understood".

3.1.2 Advantages

- Hands-free operation for the supported commands.
- Transparent rules that are easy to read, test and extend.
- Runs on ordinary hardware with only a browser, Python and Node.js; all tools are open source.
- Clear separation of frontend and backend.
- Feedback both on screen and by voice.

3.2 Proposed Methodology

Requirement analysis. Common desktop tasks (opening applications and utilities, searching the web) were listed and reduced to the nine commands of Table 3.3, together with the functional and non-functional requirements of Section 3.3.

Design. The architecture was split into a browser client, a Flask backend and the host computer (Chapter 4). React was chosen for a responsive interface [14], Flask for a small REST service [15], the Web Speech API to avoid training a recogniser, and pyttsx3 for spoken replies. The interface was kept to a single page with one microphone control.

Implementation and testing. The modules (voice capture, interpretation, execution, response) were built incrementally. Testing has two levels: module-level checks and end-to-end system tests, documented in Chapter 5.

3.3 System Analysis

3.3.1 Study of Existing Systems

Mainstream assistants are built around phones and smart speakers. On Windows, Voice Access offers extensive voice control from Windows 11 22H2 [4], while the Cortana app has been discontinued [5]. Table 2.2 compares these tools with VOICE.

3.3.2 Problems and Weaknesses of Existing Systems

- Routine desktop actions need several manual steps.
- Voice tools are tied to a vendor or an operating-system version.
- Command sets are fixed and cannot be extended by the user.
- Internal working is not open to inspection, which limits learning and customisation.

3.3.3 Functional Requirements

Table 3.1: Functional requirements and their use-case and test-case mapping

ID	Requirement	Use case	Test case(s)
FR-1	Capture speech from the microphone when the user clicks the microphone button.	Give voice command	TC-01, TC-02
FR-2	Convert speech to text and show the recognised text on screen.	Recognise speech	TC-03, TC-15
FR-3	Send the recognised text to the backend and receive a JSON response.	Give voice command	TC-03, TC-18

ID	Requirement	Use case	Test case(s)
FR-4	Identify the intent by keyword/pattern rules, tolerating filler words and verb variants (open, launch, start).	Identify intent	TC-12, TC-13
FR-5	Open Notepad, Calculator, File Explorer and Google Chrome.	Open application	TC-03 to TC-06
FR-6	Launch Task Manager and Windows Settings.	Launch system utility	TC-07, TC-08
FR-7	Search Google for a spoken query; open YouTube and Gmail.	Perform web search	TC-09 to TC-11, TC-17
FR-8	Display the response text and speak it.	Receive text and spoken response	TC-19
FR-9	For unsupported or unclear input, no speech, or a missing backend, show a clear message and perform no action.	Identify intent	TC-14 to TC-16, TC-18
FR-10	Accept consecutive commands in one session.	Give voice command	TC-20

3.3.4 Non-Functional Requirements

Table 3.2: Non-functional requirements

ID	Requirement
NFR-1 Usability	A single-page interface with one microphone control and readable status messages (a simple user experience, UX).
NFR-2 Performance	Design target: response shown within 2 seconds of the end of the utterance. This target has not been measured.
NFR-3 Reliability	Unsupported or unclear input must not crash the system or trigger an action.
NFR-4 Security	Only predefined actions are executed; text is not passed to a shell; intended for local single-user use, with no authentication.
NFR-5 Portability	Frontend runs in Chrome or Edge; command actions are tested on Windows only.
NFR-6 Maintainability	Intent rules are kept in one place so commands can be added without changing the UI.

3.3.5 Features of the System

- Voice capture and speech-to-text in the browser.
- Rule-based intent recognition with parameter extraction.
- Opening of applications, system utilities and web pages, and Google search by voice.
- Spoken and on-screen responses, and clear messages for errors.

3.3.6 Supported Voice Commands

Table 3.3: Supported voice commands of the prototype

No.	Example utterance	Action
1	“open notepad”	Launches Notepad
2	“open calculator”	Launches Calculator
3	“open file explorer”	Launches File Explorer
4	“open chrome”	Launches Google Chrome
5	“open task manager”	Launches Task Manager
6	“open settings”	Opens Windows Settings
7	“search python tutorials on google”	Opens Google results for the spoken query
8	“open youtube”	Opens youtube.com
9	“open gmail”	Opens mail.google.com

3.4 System Requirement Study

3.4.1 User Characteristics

The only user role is the normal user: a person with basic computer skills who speaks English, uses Chrome or Edge on Windows and has a working microphone. No technical knowledge is needed. The prototype has no administrator role.

3.4.2 Hardware Requirements

Table 3.4: Hardware requirements (single computer running both frontend and backend)

Item	Minimum	Recommended
Processor	Dual-core, 2 GHz	Intel Core i5 or equivalent
RAM	4 GB	8 GB
Free disk space	1 GB (Python packages and Node modules)	2 GB
Display	1366 × 768	1920 × 1080
Microphone and speakers	Built-in	Headset with microphone
Internet	Required (browser speech recognition)	Stable broadband or Wi-Fi

3.4.3 Software Requirements and Technology Stack

Table 3.5: Technology stack and versions

Layer	Technology	Purpose
Operating system	Windows 10 / 11	Primary tested OS
Backend language	Python 3	Backend logic
Web framework	Flask [15]	REST API
Text-to-speech	pyttsx3 [13]	Spoken replies (single final TTS engine)
Frontend	React, HTML, CSS, JS [14]	User interface
Runtime	Node.js and npm	Run and build the frontend
Speech-to-text	Web Speech API [8]	Speech to text in the browser
Browser	Google Chrome / Microsoft Edge	Run the frontend
Tools	Visual Studio Code; Git and GitHub	Development and version control

3.4.4 Operating System Support

Table 3.6: Operating system support status

Operating system	Status
Windows 10 / 11	Primary: developed and tested
macOS	Not tested; future work (application names differ)
Linux	Not tested; future work

CHAPTER 4 – DESIGN AND IMPLEMENTATION

4.1 System Architecture

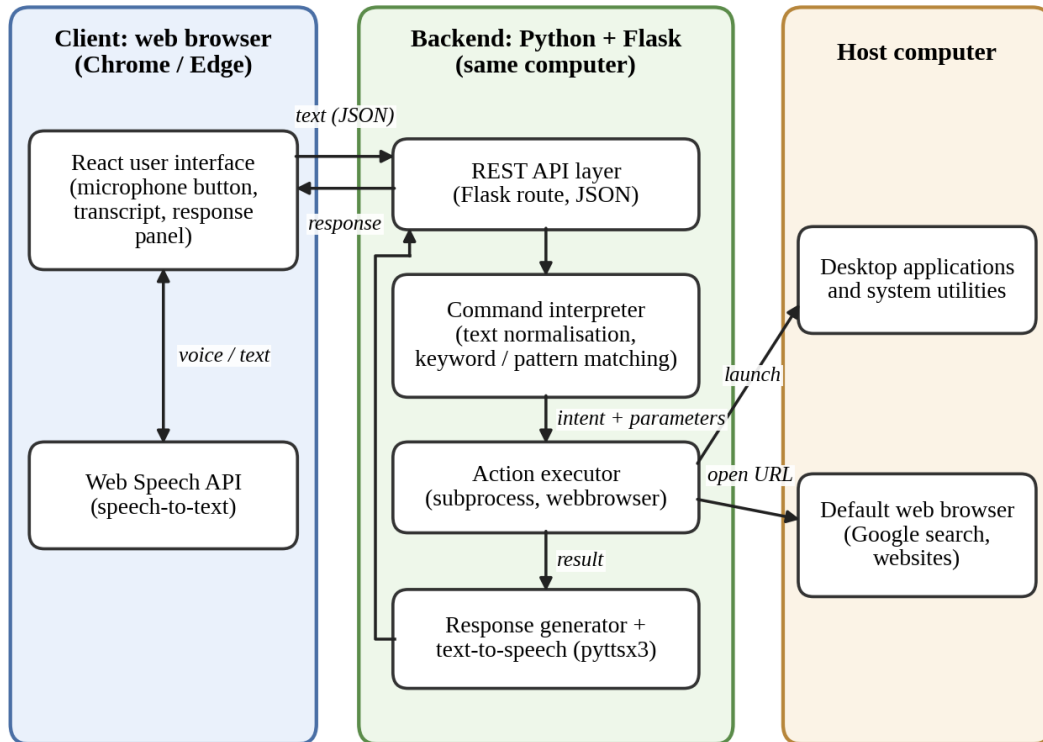


Figure 4.1: System architecture of VOICE

Figure 4.1 shows the three parts of the system. The client is a web browser running the React interface. It offers the microphone button, shows the recognised text and the reply, and uses the Web Speech API to turn speech into text [8], [14]. The recognised text is sent as JSON to the backend.

The backend is a Python Flask application running on the same computer [15]. Its REST layer receives the request and passes the text to the command interpreter, which normalises it and matches keywords and patterns to produce an intent and parameters. The action executor then carries out the predefined action, using subprocess for applications and utilities and webbrowser for web pages [12].

The result goes to the response generator, which prepares a message and speaks it with pyttsx3 [13]; the same message is returned to the browser as a JSON response. The host computer is the target of the actions: desktop applications and system utilities, and the

default web browser for searches and websites. Because the backend runs locally, it needs no separate server hosting.

4.2 Use-Case Diagram

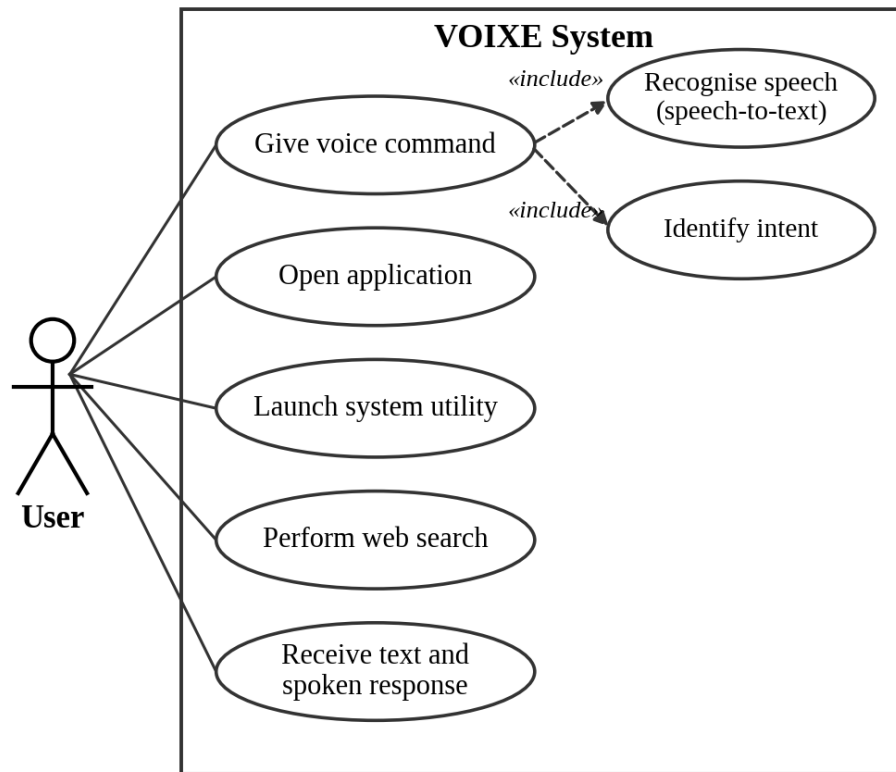


Figure 4.2: Use-case diagram of VOICE

Figure 4.2 shows the single actor, the User, and the functions the prototype offers. The user gives a voice command, opens an application, launches a system utility, performs a web search and receives a text and spoken response. “Recognise speech” and “Identify intent” are included in “Give voice command”, because every command passes through both steps.

The diagram contains only features that exist in the prototype. Login and sign-up, an administrator role, system settings, command history and help/support were shown in an earlier draft but are not implemented, so they were removed; they appear as future work in Chapter 7. Each use case is linked to functional requirements and test cases in Table 3.1.

4.3 Activity Diagram

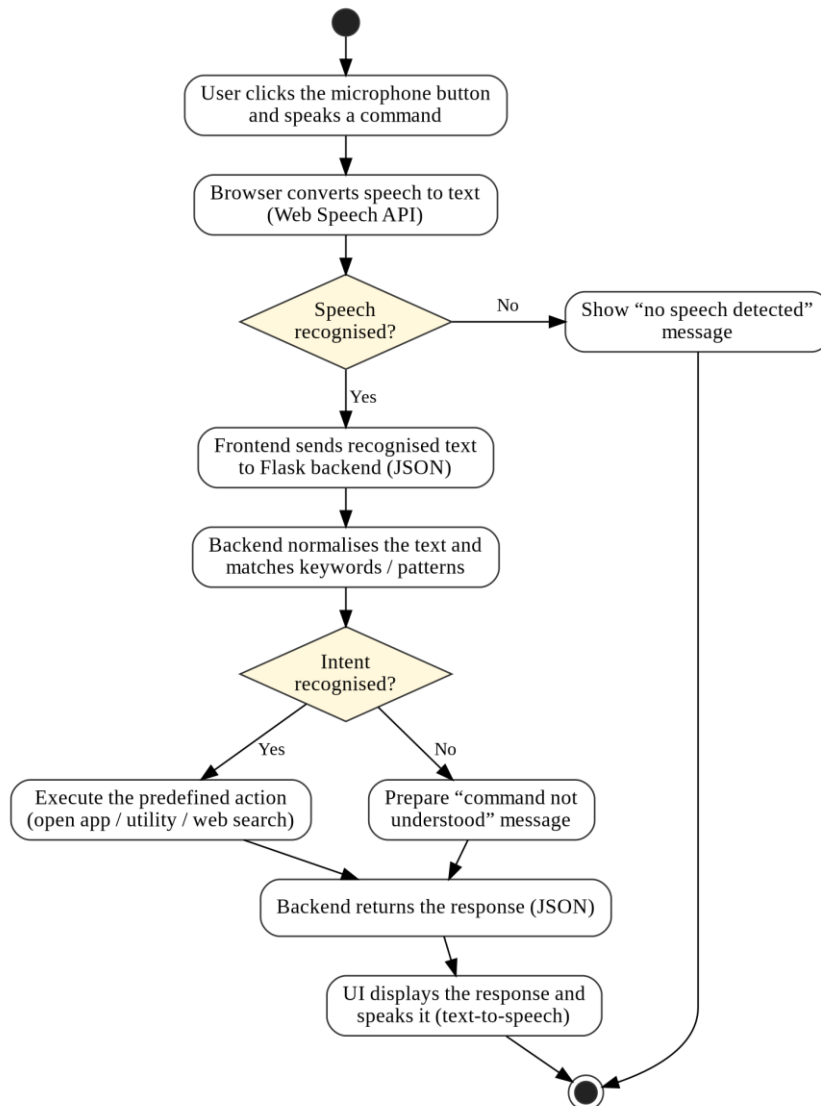


Figure 4.3: Activity diagram of a voice command

Figure 4.3 follows one command from start to end. The user clicks the microphone button and speaks, and the browser converts the speech to text. The first decision checks whether speech was recognised; if not, a “no speech detected” message is shown and the flow ends.

Otherwise the frontend sends the text to the backend, which normalises it and matches it against the rules. The second decision checks whether the intent was recognised. If yes, the predefined action is executed; if no, a “command not understood” message is prepared and nothing is executed.

Both branches rejoin: the backend returns a JSON response, and the UI displays it and speaks it. Every path ends in a defined state, so unsupported input never leaves the system waiting or triggers an unintended action (requirement FR-9).

4.4 Sequence Diagram

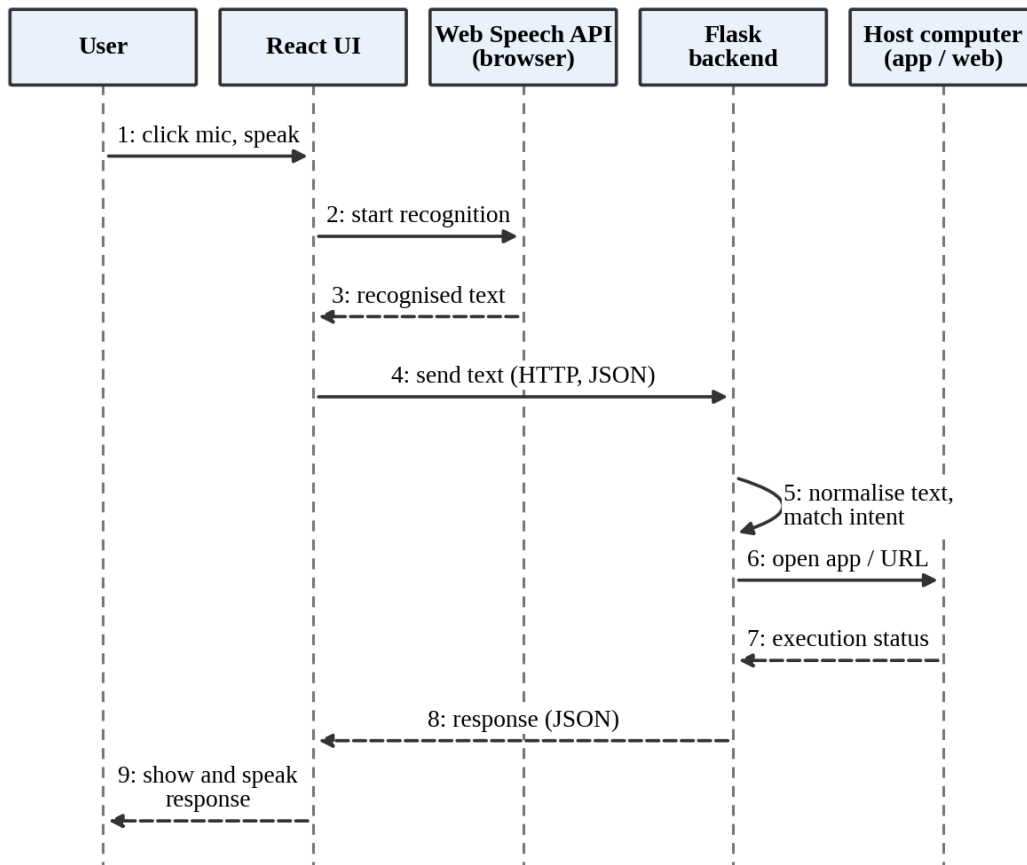


Figure 4.4: Sequence diagram of the command flow

Figure 4.4 shows the order of messages between the five participants. The user clicks the microphone and speaks (1). The React UI starts recognition in the Web Speech API (2), which returns the recognised text (3).

The UI sends the text to the Flask backend over HTTP as JSON (4). The backend normalises the text and matches the intent internally (5), then asks the host computer to open the application or URL (6) and receives the execution status (7).

The backend returns the response message as JSON (8), and the UI shows it and speaks it to the user (9). Dashed arrows are return messages. The diagram makes clear that speech recognition happens in the browser and that the backend never handles raw audio.

4.5 Data Flow Diagrams

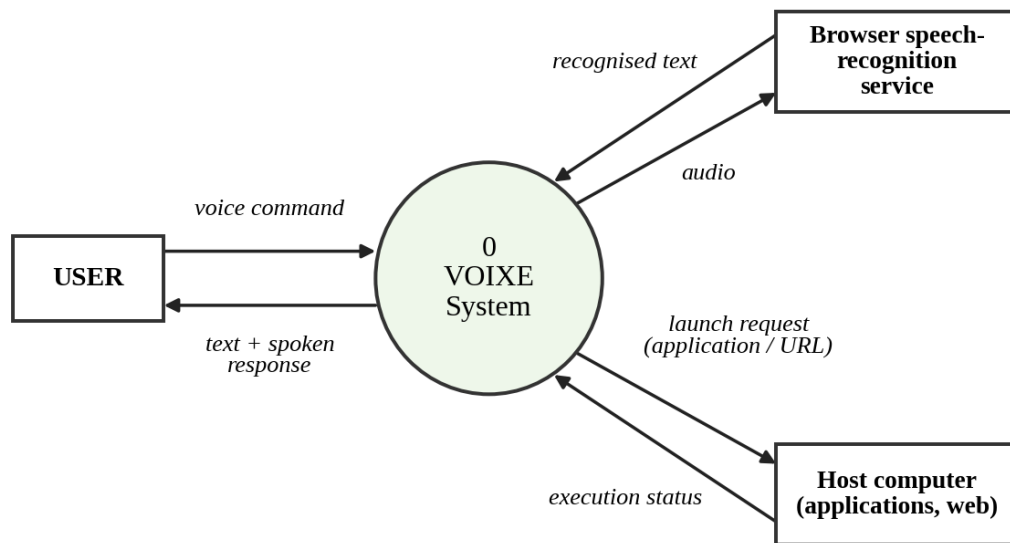


Figure 4.5: Data flow diagram, level 0

Figure 4.5 shows VOICE as one process (0) with three external entities. The user sends a voice command and receives a text and spoken response. The browser speech-recognition service receives audio and returns recognised text. The host computer receives a launch request for an application or URL and returns an execution status.

No data is stored permanently: the prototype has no database, so the diagram has no data store at this level. Level 1 in Figure 4.6 shows the internal processes.

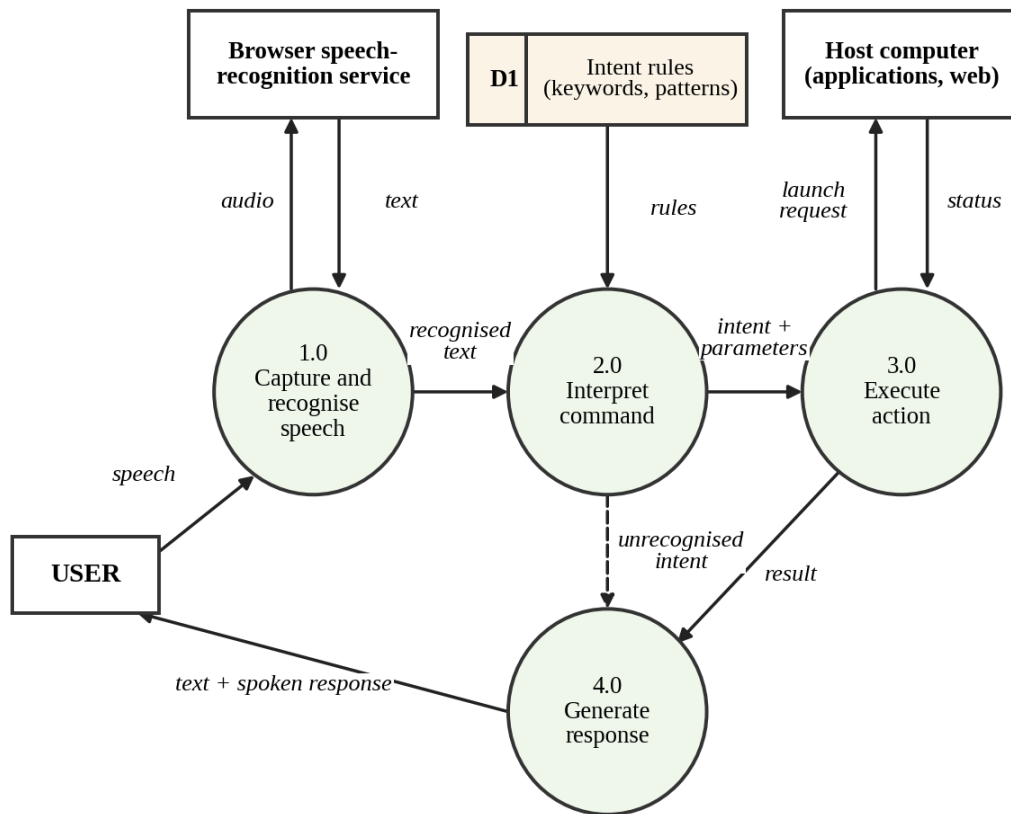


Figure 4.6: Data flow diagram, level 1

Process 1.0 captures speech and obtains recognised text from the recognition service. Process 2.0 interprets the command using the intent rules held in the code (data store D1) and produces an intent with parameters. Process 3.0 executes the action on the host computer and receives a status.

Process 4.0 generates the response from the result. When 2.0 cannot identify an intent, it passes an “unrecognised intent” flow directly to 4.0, which produces the “command not understood” message. The response returns to the user as text and speech.

4.6 Command Interpretation

VOICE uses **rule-based** interpretation, not a trained model. The backend lower-cases the text, removes punctuation and filler words (for example “please”, “the”), looks for a verb keyword (open, launch, start, search) and a target keyword (an application, utility or site name), and maps the pair to an intent. For search, the words between “search” and “on google” become the query. Table 4.1 gives worked examples (intent labels are illustrative).

Table 4.1: Worked examples from utterance to action

Spoken utterance	Normalised text	Intent and parameters	Action and reply
“open notepad”	open notepad	OPEN_APPLICATION; app = notepad	Launch Notepad; “Opening Notepad”
“Please open the calculator”	open calculator	OPEN_APPLICATION; app = calculator	Launch Calculator; “Opening Calculator”
“search python tutorials on google”	search python tutorials on google	WEB_SEARCH; query = python tutorials	Open Google results; “Searching Google for python tutorials”
“book a flight to Delhi”	book flight delhi	UNKNOWN	No action; “Sorry, I did not understand that command”

The approach is easy to test and extend, but it cannot handle free phrasing or recover from a misrecognised word; learned models such as those reviewed in Chapter 2 [3], [11] are a future direction.

4.7 Implementation Environment

Development was done in Visual Studio Code with Git and GitHub for version control. The hardware and software requirements are given in Tables 3.4 and 3.5; the primary environment is Windows with Google Chrome.

4.8 Front-End Implementation

The frontend is a single-page React application [14] written with HTML, CSS and JavaScript. It contains a microphone control, an area showing the recognised text and an area showing the system’s response. React components keep the interface state (listening, processing, result) and update the screen without reloading the page. The Web Speech API supplies the recognised text [8], which the frontend sends to the backend as a JSON request and then displays the JSON reply.

4.9 Back-End Implementation

The backend is a Python Flask application [15]. It receives the JSON request, calls the interpreter, runs the action and returns a JSON object with the intent, the response message and a status flag. Table 4.2 lists its components.

Table 4.2: Backend components and responsibilities

Component	Responsibility	Library
REST API layer	Receive the JSON request and return the JSON response	Flask
Command interpreter	Normalise text; match keywords and patterns	Python standard library
Action executor	Launch applications, utilities and URLs	subprocess, webbrowser [12]
Response generator	Build the reply and speak it	pyttsx3 [13]

4.10 User Interface

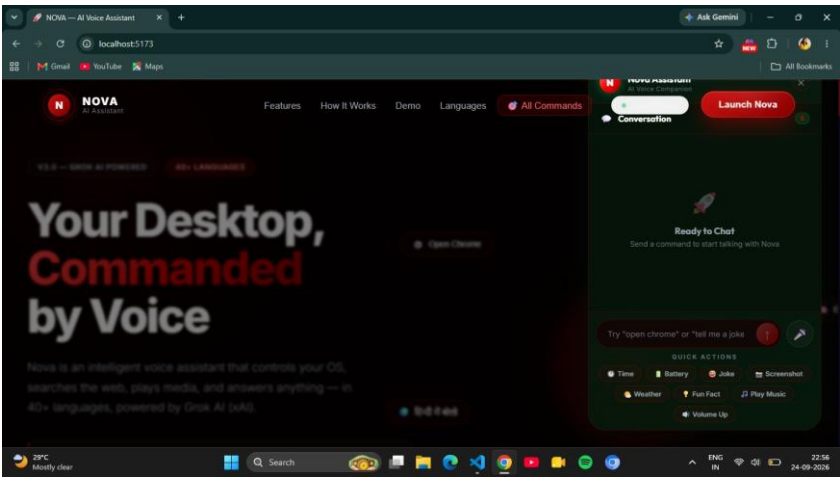


Figure 4.7: Main screen of the VOICE interface

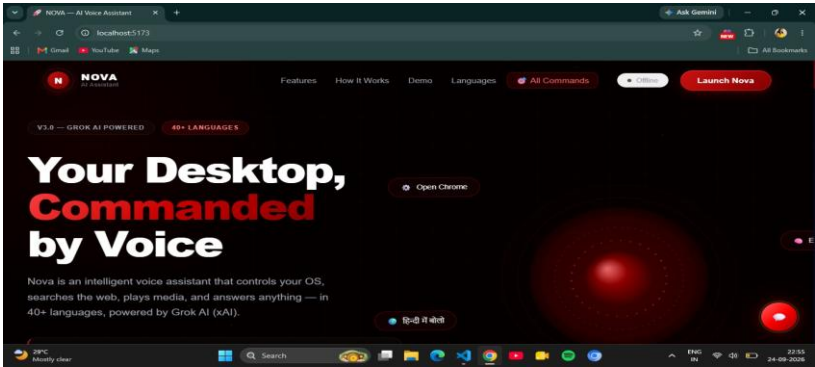


Figure 4.8: Recognised command and system response

Figure 4.7 shows the initial screen, where the user clicks the microphone button to start listening. Figure 4.8 shows a command after recognition, with the recognised text and the response displayed together.

4.11 Safety Measures in the Implementation

- Only the predefined actions of Table 3.3 are executed; user text is never passed to a shell.
- Empty or unrecognised text produces a message, not an action.
- The Flask development server listens on the local machine by default, so the service is not exposed to other computers.

CHAPTER 5 – TESTING AND DEPLOYMENT

5.1 Testing Approach

Testing was organised at two levels [7]. Module-level checks covered speech capture, intent matching, action execution and text-to-speech individually. End-to-end system tests then ran complete voice commands through the interface, as listed in Table 5.2. Each test case has a unique ID, steps, an expected result, the actual result and a pass/fail verdict. User-acceptance, performance and security testing were not performed and are listed as future work.

5.2 Test Environment

Table 5.1: Test environment

Item	Details
Computer	8GB or 16GB RAM
Operating system	[Windows 10 / 11, build]
Browser	[Google Chrome, version]
Microphone	[built-in / headset]
Network	[Wi-Fi / broadband]
Python / Node.js	[3.12]
Room condition	Quiet room

5.3 Test Cases

The Actual result and Pass/Fail columns must be filled in after executing each test on the machine described in Table 5.1.

Table 5.2: Test cases for the VOICE prototype

Test ID	Steps / input	Expected result	Pass / Fail
TC-01	Allow microphone; click the microphone button.	Status shows “Listening...”.	Pass
TC-02	Block microphone permission; click the button.	A clear error message is shown; no crash.	Pass
TC-03	Say “open notepad”.	Recognised text is shown; Notepad opens; reply “Opening Notepad”.	Pass

Test ID	Steps / input	Expected result	Pass / Fail
TC-04	Say “open calculator”.	Calculator opens; reply is shown.	Pass
TC-05	Say “open file explorer”.	File Explorer opens; reply is shown.	Pass
TC-06	Say “open chrome”.	Google Chrome opens; reply is shown.	Fail
TC-07	Say “open task manager”.	Task Manager opens; reply is shown.	Pass
TC-08	Say “open settings”.	Windows Settings opens; reply is shown.	Pass
TC-09	Say “search python tutorials on google”.	Browser opens Google results for “python tutorials”.	Pass
TC-10	Say “open youtube”.	youtube.com opens in the browser.	Pass
TC-11	Say “open gmail”.	mail.google.com opens in the browser.	Pass
TC-12	Say “please open the calculator”.	Filler words ignored; Calculator opens.	Pass
TC-13	Say “launch notepad”.	Verb variant accepted; Notepad opens.	Pass
TC-14	Say “book a flight to Delhi”.	“Command not understood” reply; no action.	Pass
TC-15	Click the button and stay silent for about 5 seconds.	“No speech detected” message; no request sent.	Fail
TC-16	Say “open photoshop” (not supported).	Unsupported-command message; no crash.	Fail
TC-17	Say “search on google” without a query.	System asks for a search term; no empty search.	Pass
TC-18	Stop the backend; give any command.	UI shows a connection-error message.	Pass
TC-19	Repeat TC-03 with speakers on.	The reply is spoken aloud.	Pass
TC-20	Say “open calculator”, then “open notepad”.	Both commands execute in order.	Pass

5.4 Test Result Summary

[After execution: “N of 20 test cases passed; failed cases:6 fix applied:6. No accuracy percentage or response-time figure is claimed in this report, because no measured dataset was collected.

5.5 Deployment

VOICE is deployed locally on the user's computer, not on a public server. The steps are:

- Install Python and Node.js.
- Install the backend packages (Flask, pytsx3) and the frontend dependencies (npm install).
- Start the Flask backend.
- Start the React frontend (development server or production build).
- Open the page in Chrome or Edge and allow microphone access.
- If frontend and backend use different ports, enable cross-origin requests in the backend.

5.6 Maintenance

Bugs found during use are fixed in the source code. New commands are added by extending the rule set in the backend and adding a test case to Table 5.2.

CHAPTER 6 – ANALYSIS AND RESULTS

6.1 Introduction

This chapter analyses what the prototype does, compares it with an existing tool, and states its limitations. Only claims supported by the design and the test cases are made. No graphs are included because no measured data, such as accuracy or response times, was collected.

6.2 Functional Analysis

The prototype implements the complete path of Figure 4.3: speech capture, speech-to-text in the browser, JSON communication with the backend, rule-based intent recognition, execution of the nine commands of Table 3.3, and a spoken and on-screen reply. Unsupported input produces a message and no action. Whether each function behaves as expected is recorded per test case in Table 5.2.

6.3 Comparison with an Existing Tool

Table 6.1: Comparison of Windows Voice Access and VOICE

Feature	Windows Voice Access	VOICE (this prototype)
Platform	Windows 11, version 22H2 or later [4]	Browser frontend; command actions tested on Windows only
Scope of control	Opening and switching apps, browsing, mail and text [4]	Nine predefined commands
Speech recognition	On-device; works without internet after set-up [4]	Browser service; Chrome sends audio to a server, so internet is needed [8]
Customisation	Fixed by the vendor	Rules can be edited in the source code
Maturity	Released product	Prototype tested with 20 test cases
Accessibility	Designed for accessibility [4]	Potential benefit only; not tested with users

Cortana is not compared because its standalone Windows app is no longer supported [5]. VOICE is not a replacement for Voice Access. Its value lies in being small, transparent and extensible, which suits learning and experimentation.

6.4 Limitations

- Recognition depends on the browser's service; it needs internet in Chrome [8] and its behaviour with accents or noise has not been measured.
- Only nine English command patterns are understood; free phrasing is not.
- Actions are Windows-specific; macOS and Linux are untested.
- No authentication, user accounts or command history.
- No quantitative evaluation of accuracy or response time has been done.

6.5 Accessibility

Hands-free control may benefit people with limited mobility. This is a **potential** benefit only: the prototype has not been tested with such users, so no accessibility improvement is claimed.

6.6 Done and Planned

Table 6.2: Status of features

Feature	Status
Browser speech-to-text; rule-based intents (9 patterns); spoken and on-screen reply	Implemented
Test plan with 20 test cases	Defined; results recorded in Table 5.2
Measured accuracy and response time	Not done (Chapter 7)
Offline recognition, learned NLU, multi-language support	Future work (Chapter 7)
Login, administrator module, command history	Future work (Chapter 7)
macOS and Linux support	Future work (Chapter 7)

CHAPTER 7 – CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

VOICE demonstrates that a browser-based interface, a small Flask backend and a rule-based interpreter can turn spoken commands into desktop and web actions with spoken feedback. The prototype implements the full voice-to-action path for nine commands, handles unsupported input safely, and is documented with requirements, diagrams and a 20-case test plan. Its limits are equally clear: it understands a fixed English command set, depends on the browser's online recognition service, is tested on Windows only, and has no measured accuracy or response-time figures. The work should therefore be seen as a working foundation, not a finished product.

7.2 Future Enhancements

- **Measured evaluation:** collect a defined set of utterances and report word error rate, command success rate and response time, followed by a user-acceptance study.
- **Learned language understanding:** replace or complement the rules with an intent and slot model, guided by work such as [3], [11].
- **Offline recognition:** use an on-device recogniser to remove the dependency on internet access [8], [4].
- **Multi-language support:** extend to other languages, using multilingual data such as [11].
- **Dialogue and wake word:** add a wake word, follow-up questions and context across turns.
- **Accounts, history and security:** add authentication, an administrator module and command history with a database.
- **Cross-platform support:** add macOS and Linux actions and evaluate accessibility with real users.

CHAPTER 8 – REFERENCES

- [1] M. Saebi, E. Pusateri, A. Meghawati, and C. Van Gysel, “A discriminative entity-aware language model for virtual assistants,” in *Proc. Interspeech*, Brno, Czechia, 2021, pp. 2032–2036, doi: 10.21437/Interspeech.2021-1767.
- [2] V. Mitra *et al.*, “Analysis and tuning of a voice assistant system for dysfluent speech,” in *Proc. Interspeech*, Brno, Czechia, 2021, pp. 4848–4852.
- [3] M. Radfar, A. Mouchtaris, S. Kunzmann, and A. Rastrow, “FANS: Fusing ASR and NLU for on-device SLU,” in *Proc. Interspeech*, Brno, Czechia, 2021, arXiv:2111.00400.
- [4] Microsoft, “Set up voice access,” Microsoft Support. [Online]. Available: <https://support.microsoft.com/en-US/Accessibility/windows/voice-access/set-up-voice-access> (accessed Sep. 2026).
- [5] Microsoft, “End of support for Cortana in Windows,” Microsoft Support. [Online]. Available: <https://support.microsoft.com/en-us/topic/end-of-support-for-cortana-in-windows-d025b39f-ee5b-4836-a954-0ab646ee1efa> (accessed Sep. 2026).
- [6] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2009.
- [7] R. S. Pressman, *Software Engineering: A Practitioner’s Approach*, 7th ed. New York, NY, USA: McGraw-Hill, 2010.
- [8] MDN Web Docs, “SpeechRecognition,” Mozilla. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/SpeechRecognition> (accessed Sep. 2026).
- [9] D. Muralidharan *et al.*, “DEXTER: Deep encoding of external knowledge for named entity recognition in virtual assistants,” in *Proc. Interspeech*, Brno, Czechia, 2021, pp. 1234–1238, doi: 10.21437/Interspeech.2021-1877.
- [10] C. Van Gysel, M. Hannemann, E. Pusateri, Y. Oualil, and I. Oparin, “Space-efficient representation of entity-centric query language models,” in *Proc. Interspeech*, Incheon, South Korea, 2022, pp. 679–683.
- [11] J. FitzGerald *et al.*, “MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages,” in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, Toronto, Canada, Jul. 2023, pp. 4277–4302, doi: 10.18653/v1/2023.acl-long.235.
- [12] Python Software Foundation, “The Python standard library: subprocess and webbrowser modules.” [Online]. Available: <https://docs.python.org/3/library/> (accessed Sep. 2026).
- [13] “pyttsx3: Text-to-speech conversion library in Python,” Python Package Index (PyPI). [Online]. Available: <https://pypi.org/project/pyttsx3/> (accessed Sep. 2026).